

03. Travel Agent - API di base

3. Implementazione dell'API Base: Guida Dettagliata

3.1 Il File `main.py`

Il file `main.py` è il punto di ingresso principale dell'applicazione.

Crea il file nella cartella `travel_agent_api`

Analizziamo ogni componente in dettaglio:

```
# Questa riga importa la classe FastAPI dal framework FastAPI. È l'elemento principale
necessario per creare un'applicazione web API.
from fastapi import FastAPI

# Qui viene importato un router chiamato `chat_router` da un modulo locale (file)
`app.routes`. I router in FastAPI sono utilizzati per organizzare le diverse route (endpoint)
dell'API in modo modulare.
from travel_agent_api.routes import chat_router

# Questa riga importa il middleware CORS (Cross-Origin Resource Sharing). CORS è un
meccanismo di sicurezza che consente o blocca le richieste da domini diversi da quello
dell'API.
from fastapi.middleware.cors import CORSMiddleware

# Qui viene creata un'istanza dell'applicazione FastAPI. Questo oggetto `app` sarà il punto
principale dell'applicazione dove verranno registrate tutte le route e i middleware.
app = FastAPI()
```

Configurazione CORS

Il Cross-Origin Resource Sharing (CORS) è un meccanismo che permette a risorse di essere richieste da un dominio diverso da quello che serve la risorsa.

```
origins = [
    "http://127.0.0.1:8000", # Porta standard dell'applicazione front-end Laravel
    "http://localhost:8000" # Alias per localhost e Porta standard dell'applicazione
    front-end Laravel
]
```

La configurazione del middleware CORS:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,          # Domini permessi
    allow_credentials=True,         # Permette l'invio di credenziali
    allow_methods=["*"],            # Permette tutti i metodi HTTP
    allow_headers=["*"],            # Permette tutti gli headers
)
```

Il CORS (Cross-Origin Resource Sharing) è un meccanismo di sicurezza implementato nei browser web. Facciamo un esempio concreto:

Mettiamo che tu stia sviluppando un'applicazione web front-end che gira su `http://localhost:8000` e hai un'API back-end su `http://localhost:8080`. Quando il tuo front-end prova a fare una richiesta AJAX/fetch al

back-end, il browser bloccherà la richiesta a meno che il back-end non specifichi esplicitamente che accetta richieste da quel dominio.



Il CORS è fondamentale per la sicurezza web perché:

1. Impedisce a siti malevoli di fare richieste non autorizzate ai tuoi endpoint
2. Ti permette di specificare esattamente quali domini possono interagire con la tua API
3. Puoi anche configurare quali metodi HTTP sono permessi (GET, POST, ecc.)

È come un firewall a livello applicativo: invece di bloccare tutto il traffico verso un IP, blocca le richieste web in base alla loro origine.

Se stai sviluppando API RESTful o applicazioni che usano microservizi, dovrai quasi sicuramente configurare il CORS correttamente per far funzionare la tua applicazione.

Inclusione del Router

Nel contesto delle API, un router è un componente che gestisce l'instradamento delle richieste HTTP verso le appropriate funzioni o handler che le gestiranno. Il router analizza il percorso (path) dell'URL della richiesta, il metodo HTTP utilizzato (GET, POST, PUT, DELETE, ecc.) e qualsiasi parametro presente, indirizzando poi la richiesta alla corretta logica di business. Ad esempio, quando arriva una richiesta GET a `/api/users`, il router la indirizzerà alla funzione che gestisce il recupero degli utenti, mentre una richiesta POST a `/api/users` verrà instradata alla funzione che gestisce la creazione di un nuovo utente. Questo sistema di routing è fondamentale per organizzare la logica del backend in modo modulare e mantenibile, permettendo una chiara separazione delle responsabilità all'interno dell'applicazione.

```
app.include_router(  
    chat_router.router,    # Il router definito in chat_router.py  
    tags=["Chat"],         # Tag per la documentazione Swagger  
    prefix="/chat"         # Prefisso per tutte le route del router  
)
```

4. Implementazione del Router

4.1 Struttura del Router

Crea la cartella `routes` all'interno di `travel_agent_api` e poi crea in tale cartella il file `chat_router.py` dovresti avere una struttura di cartelle come questa:

```
├─ README.md  
├─ poetry.lock  
├─ pyproject.toml  
├─ tests  
├─ __init__.py  
└─ travel_agent_api  
    ├── __init__.py  
    ├── main.py  
    └── routes  
        └─ chat_router.py
```

Il file `routes/chat_router.py` definisce le route specifiche per la gestione della chat.

```
# chat_router.py  
from fastapi import APIRouter  
from pydantic import BaseModel  
from travel_agent_api.services.agent_service import Agent
```

```
# Creazione di un nuovo router FastAPI
router = APIRouter()
```

4.2 Definizione del Modello di Richiesta

Questo codice definisce una classe chiamata `ChatCompletionRequest` che serve a strutturare e validare le richieste di chat.

La parte `model_config` serve per mostrare nell'interfaccia di test (`/doc`) un esempio di chiamata.

```
# Definizione del modello di dati per la richiesta di chat
class ChatCompletionRequest(BaseModel):
    # Il filed messages conterra' la lista dei messaggi che compongono la conversazione
    messages: list

    # un esempio di utilizzo
    # utile perche' verra' visualizzato come esempio nella schermata di test in /docs
    model_config = {
        "json_schema_extra": {
            "example": {
                "messages": [
                    {
                        "role": "user", #
                        "content": "Vorrei organizzare un viaggio a Roma"
                    }
                ]
            }
        }
    }
```

4.3 Endpoint di Chat

```
@router.post("/travel-agent") # Definizione dell'endpoint POST per il travel agent
def chat_completion(request: ChatCompletionRequest):
    """
    Endpoint per la gestione delle richieste di chat.
    Processa i messaggi ricevuti e restituisce una risposta dall'agente di viaggio.

    Args:
        request (ChatCompletionRequest): La richiesta contenente i messaggi della
        conversazione

    Returns:
        dict: La risposta elaborata dall'agente di viaggio

    Raises:
        HTTPException: In caso di errori durante l'elaborazione della richiesta
    """

    # Creazione di una nuova istanza dell'agente
    agent = Agent()

    # Elaborazione dei messaggi e generazione della risposta
    response = agent.run(messages=request.messages)
```

```
# Tracing
print("*" * 80)
print("chat_completion")
print("*" * 80)
# print("request messages: ", request.messages)
# print("response: ", response)

# Restituzione della risposta
return response
```

Questa implementazione fornisce una base solida e sicura per la gestione delle richieste di chat, con validazione appropriata degli input e gestione degli errori.